

Chapter 4: Proposed System Architecture

4.1 Overview

The proposed V3 congestion-control system is designed as a feedback-based congestion-control framework that combines **Explicit Congestion Notification (ECN) information** with **Bandwidth-Delay Product (BDP) estimation** to dynamically regulate the TCP congestion window (CWND).

The proposed implementation consists of six functional modules:

1. **Traffic Generator**
2. **VBR Generator**
3. **ECN Monitor**
4. **ECN Controller**
5. **BDP Monitor**
6. **BDP Controller**

These six modules operate within a shared network environment containing a bottleneck link and an ECN-enabled queue. The network configuration is not considered an additional software module; it provides the environment through which congestion information is generated and fed back to the controller.

The Traffic Generator generates TCP traffic, while the VBR Generator introduces variable-rate background traffic. The resulting traffic passes through the shared bottleneck, where congestion can produce ECN feedback. The ECN Monitor collects the congestion-marking information, and the ECN Controller uses this information to determine an ECN-based CWND adjustment. In parallel, the BDP Monitor obtains RTT and bandwidth information and estimates the BDP. The BDP Controller then applies the BDP constraint to obtain the final congestion-window value.

The overall feedback mechanism can be represented as:

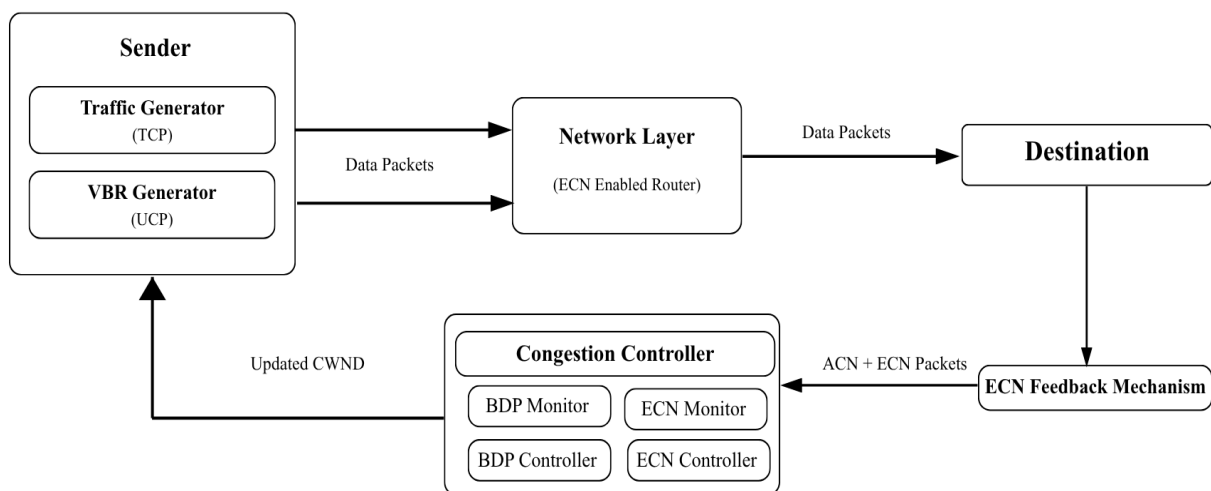


Fig. 4.1.1 High Level Architecture Diagram

4.2 Network Topology

Network Topology of the proposed Congestion-Control System:

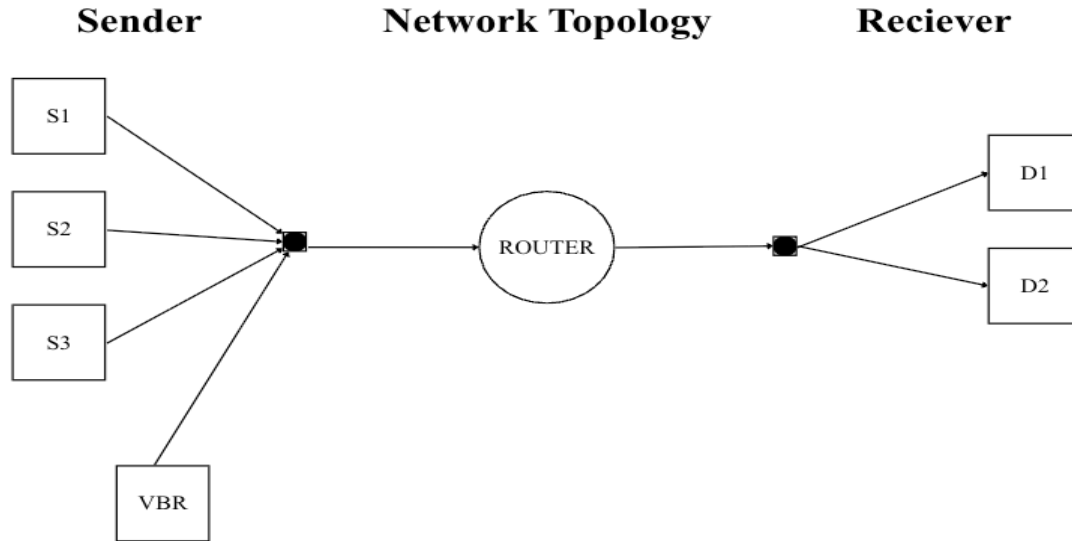


Fig 4.2.1 Network Topology of the proposed Congestion-Control System

4.3 Traffic Generator Module

The Traffic Generator module is responsible for generating the primary TCP traffic used to evaluate the proposed congestion-control mechanism.

The module configures the TCP sender and receiver and determines the amount of application data to be transmitted based on the configured packet count and packet size. The implementation calculates the total application data as:

$$D = N_p \times S_p$$

where:

- DD = total application data in bytes
- N_p = number of application packets
- S_p = packet size in bytes.

Algorithm 1: TCP Traffic Generation

Input: Packet count N_p , Packet size S_p , Source node, Destination node Simulation time T.

Output: TCP traffic

Begin

1. Initialize TCP sender and receiver.
2. Read packet count N_p and packet size S_p .

3. Calculate total data:

$$D = N_p \times S_p$$

4. Configure TCP sender.

5. Configure TCP receiver.

6. Assign source and destination addresses.

7. Start TCP transmission.

8. Continue transmission until D bytes are transmitted or simulation time expires.

9. Record transmission statistics.

End

4.4 VBR Generator Module

The VBR Generator provides variable-rate background traffic to create a dynamic network load. This prevents the congestion environment from remaining completely static and allows the proposed controller to operate under changing traffic conditions.

The VBR traffic rate varies between a minimum and maximum configured rate.

The instantaneous rate can be represented as:

$$R_{VBR}(t) \in [R_{min}, R_{max}]$$

where:

- $R_{VBR}(t)$ = VBR transmission rate at time t
- R_{min} = minimum VBR rate
- R_{max} = maximum VBR rate.

The VBR generator is installed separately from the TCP traffic generator and uses UDP-based traffic in the simulation.

Algorithm 2: VBR Traffic Generation

Input: Minimum rate R_{min} , Maximum rate R_{max} , Packet size S_p , Simulation time T

Output: Variable-rate background traffic

Begin

1. Initialize VBR sender and receiver.
2. Configure UDP transport.
3. Set minimum transmission rate R_{min} .
4. Set maximum transmission rate R_{max} .
5. Select/update the current VBR rate.
6. Generate packets according to the current rate.
7. Transmit packets through the shared network.
8. Repeat rate generation/update until simulation ends.

End.

4.5 ECN Monitor Module

The ECN Monitor is responsible for collecting congestion information from the network. It maintains the number of packets observed and the number of packets marked with ECN.

The module maintains the following parameters:

- Total packets
- ECN-marked packets
- ECN ratio
- ECN acknowledgement count
- Last ECN acknowledgement time

The implementation connects the queue's Enqueue and Mark trace sources to the ECN Monitor.

Mathematical Formulation

The cumulative ECN ratio is:

$$E = \frac{M}{P}$$

where:

- EE = ECN ratio
- MM = number of ECN-marked packets
- PP = total number of observed packets.

The ECN percentage is:

$$E_{\%} = E \times 100$$

Algorithm 3: ECN Monitoring

Input: Packet enqueue events, ECN marking events, ECN acknowledgement events

Output: ECN ratio E

Begin

1. Initialize total packet counter $P = 0$.
2. Initialize marked packet counter $M = 0$.
3. Initialize ECN ACK counter.
4. When a packet enters the queue:
 $P = P + 1$
5. When a packet is ECN marked:
 $M = M + 1$
6. Calculate:
 $E = M / P$
7. When an ECN ACK is received:

Increment ECN ACK counter.

8. Record the latest ECN ACK time.

9. Provide E to the ECN Controller.

End.

4.6 ECN Controller Module

The ECN Controller is the primary congestion-response component of the V3 system. It uses the ECN ratio obtained from the ECN Monitor to determine how strongly the congestion window should be adjusted.

The controller uses the configurable parameter α to control the sensitivity of CWND reduction to the observed ECN ratio.

The core equation implemented by the controller is:

$$CWND_{new} = CWND_{old}(1 - \alpha E)$$

where:

- $CWND_{old}$ = previous congestion window
- $CWND_{new}$ = ECN-adjusted congestion window
- α = ECN response parameter
- E = ECN ratio.

Algorithm 4: ECN-Based CWND Adaptation

Input: Previous CWND, ECN ratio E, Alpha α

Output: ECN-adjusted CWND

Begin

1. Obtain the current CWND.
2. Obtain the current ECN ratio E.
3. Read the controller parameter α .
4. Calculate the congestion reduction factor:

$$F = \alpha \times E$$

5. Calculate the new CWND:

$$CWND_{new} = CWND_{old} \times (1 - F)$$

6. Store the calculated CWND.
7. Pass the result to the BDP Controller.

End.

4.7 BDP Monitor Module

The BDP Monitor estimates the capacity of the network path using the measured RTT and the configured bottleneck bandwidth.

The Bandwidth-Delay Product represents the amount of data that can theoretically be in flight while fully utilizing the available bottleneck bandwidth.

The basic BDP equation used by the implementation is:

$$BDP = B \times RTT$$

When bandwidth is represented in bits per second, the BDP in bytes is:

$$BDP_{bytes} = \frac{B_{bps} \times RTT}{8}$$

The equivalent BDP in packets is:

$$BDP_{packets} = \frac{BDP_{bytes}}{S_p}$$

Algorithm 5: BDP Estimation

Input: Bottleneck bandwidth B, Measured RTT, Packet size Sp

Output: BDP in bytes and packets

Begin

1. Obtain current RTT.
2. Obtain bottleneck bandwidth B.
3. Convert bandwidth from bits/s to bytes/s.
4. Calculate:

$$BDP_{bytes} = (B \times RTT) / 8$$

5. Calculate:

$$BDP_{packets} = BDP_{bytes} / S_p$$

6. Store BDP values.
7. Provide BDP to the BDP Controller.

End.

4.8 BDP Controller Module

The BDP Controller provides the capacity constraint for the congestion window calculated by the ECN Controller.

The ECN Controller first produces an ECN-based CWND. The BDP Controller then compares this value with the estimated BDP and selects the smaller value.

The final CWND is therefore:

$$CWND_{final} = \min(CWND_{formula}, BDP)$$

where:

- $CWND_{formula}$ = CWND calculated using ECN feedback
- BDP = estimated network capacity.

Combining this with the ECN equation gives:

$$CWND_{final} = \min[CWND_{old}(1 - \alpha E), BDP]$$

Algorithm 6: BDP-Based CWND Constraint

Input: ECN-derived CWND, Estimated BDP

Output: Final CWND

Begin

1. Receive CWNDformula from ECN Controller.
2. Obtain current BDP.
3. Compare CWNDformula and BDP.
4. Select the smaller value:
$$\text{CWND}_{\text{final}} = \min(\text{CWND}_{\text{formula}}, \text{BDP})$$
5. Apply CWNDfinal to the TCP connection.
6. Record the CWND update.

End.

4.9 Complete V3 Control Algorithm

The six modules operate together as a single closed-loop mechanism.

Algorithm 7: Proposed V3 Congestion-Control Algorithm

Input: TCP traffic parameters, VBR traffic parameters, Bottleneck bandwidth B, Packet size S_p

Output: Dynamically adapted CWND

Begin

1. Initialize the Traffic Generator.
2. Initialize the VBR Generator.
3. Start TCP and VBR traffic.
4. Transmit traffic through the shared bottleneck.
5. Monitor packet enqueue events.
6. Monitor ECN marking events.
7. ECN Monitor calculates ECN ratio E .
8. Obtain RTT from the network.
9. BDP Monitor calculates:
$$\text{BDP} = B \times \text{RTT}$$
10. ECN Controller calculates:
$$\text{CWND}_{\text{formula}} = \text{CWND}_{\text{old}} \times (1 - \alpha E)$$
11. BDP Controller calculates:
$$\text{CWND}_{\text{final}} = \min(\text{CWND}_{\text{formula}}, \text{BDP})$$
12. Apply CWNDfinal to the TCP sender.
13. Continue transmission using the updated CWND.
14. Repeat the monitoring and control process whenever new feedback is received.

15. Stop when the simulation time expires.

16. Record performance metrics.

End.